

1. Introduccion

Retrieval-Augmented Generation (RAG) combina recuperacion de informacion con generacion de lenguaje. En lugar de depender solo del conocimiento interno del modelo (parametros), RAG recupera fragmentos relevantes de una base documental externa y los incorpora como contexto.

Paper fundacional: 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks' (Lewis et al., Facebook AI Research, 2020, arXiv:2005.11401). Demostro que RAG supera a modelos puramente generativos en tasks que requieren conocimiento factual: QA, verificacion de hechos, traduccion especializada.

1.1 Arquitectura RAG

Tres componentes:

- Indexador: procesa y almacena documentos en formato buscable.
- Recuperador (Retriever): selecciona fragmentos relevantes dada una consulta.
- Generador (Generator): LLM que recibe consulta + fragmentos y produce respuesta.

1.2 Implementacion en el Proyecto

knowledge.py implementa un recuperador por coincidencia de terminos (bag-of-words):

```
def retrieve_snippets(question, knowledge_base=None, limit=2):
    knowledge_base = knowledge_base or load_knowledge_base()
    terms = set(question.lower()
                .replace("?", "").replace(" ", "").split())
    scored = []
    for item in knowledge_base:
        haystack = f"{item['title']} {item['content']}".lower()
        score = sum(1 for term in terms if term in haystack)
        if score > 0:
            scored.append((score, item))
    scored.sort(key=lambda pair: pair[0], reverse=True)
    return [item for _, item in scored[:limit]]
```

Base de conocimiento actual (knowledge_base.json):

```
[
  {"id": "sdd-bdd-tdd", "title": "SDD, BDD y TDD",
   "content": "El curso usa SDD para disenar antes de construir..."},
  {"id": "rag", "title": "RAG",
   "content": "RAG recupera fragmentos de una base documental..."},
  {"id": "cag", "title": "CAG",
   "content": "CAG usa contexto persistente..."},
  {"id": "jarvis", "title": "Tony Stark y Jarvis",
   "content": "La IA debe funcionar como Jarvis..."}
]
```

2. Variantes de RAG

- RAG Secuencial: recuperar primero, generar despues (enfoque clasico, usado en el proyecto).
- RAG Iterativo: multiples ciclos recuperacion-generacion para preguntas complejas.
- RAG con Memoria: incorpora historial de conversacion en la consulta de recuperacion.
- RAG Hibrido: combina busqueda semantica (vectores/embeddings) con busqueda lexica (BM25).

El proyecto implementa RAG secuencial. Para RAG hibrido se podria integrar Ollama con embeddings (nomic-embed-text) en el recuperador.

3. Evaluacion de RAG

- Exact Match (EM): porcentaje de respuestas que coinciden exactamente con el gold standard.
- F1 Score: promedio ponderado de precision y recall a nivel de tokens.
- Faithfulness: que tan fiel es la respuesta a los fragmentos recuperados (evita alucinaciones).
- Answer Relevance: que tan relevante es la respuesta para la pregunta original.

Para este proyecto, las pruebas base verifican que la respuesta CONTenga el fragmento esperado (assertIn), validando faithfulness implicita.

4. Referencias

RAG Paper: Lewis et al. (2020) - [arXiv:2005.11401](https://arxiv.org/abs/2005.11401)

Designing Machine Learning Systems (Chip Huyen) - O'Reilly

LangChain RAG Documentation - python.langchain.com

Google Scholar - Retrieval-Augmented Generation

RAGAS (RAG Assessment) - docs.ragas.io